UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE
Programación Avanzada · Unidad 1 · Grupo 9

# Gestión de clientes
# Aporte individual

**Josué Nicolás Muñoz Gualotuña**
jnmunoz2@espe.edu.ec
12 de noviembre de 2026

## Propósito y alcance

Administrar clientes del sistema de ventas y facturación mediante una API organizada en capas. El módulo valida los datos, evita duplicados y protege el historial de ventas. La asignación en CONTRIBUTING.md corresponde a la entidad Cliente, su CRUD y sus reglas de negocio.

La base del equipo ya contenía el CRUD. Esta contribución corrige la unicidad del correo ante diferencias de mayúsculas, normaliza el correo guardado, mejora el filtro por apellido y agrega pruebas, demostración y documentación. Los componentes existentes conservan su historial de autoría.

## Repositorio y rama

github.com/sgsanchez/ProgAv-TG1-Grupo9
Rama de trabajo: feature/clientes

## Contenido de este aporte

1. Alcance y objetivos (p. 1).
2. Arquitectura y modelo de datos (p. 2).
3. Contrato HTTP y reglas (p. 3).
4. Verificación, defensa y pendientes grupales (p. 4).

**Documento de apoyo:** integrar este contenido en el informe grupal TallerGrupal1_Grupo9.pdf. No sustituye el informe conjunto ni el video de defensa.

## 2. Arquitectura y modelo de datos

La lógica de negocio permanece en ClienteService. El controlador recibe solicitudes; el repositorio accede a la base mediante Spring Data JPA; los DTOs delimitan los datos de entrada y salida.

| Flujo | Responsabilidad |
|---|---|
| HTTP -> ApiKeyInterceptor | Valida X-API-KEY antes de acceder al módulo. |
| ClienteController -> ClienteRequest | Recibe GET, POST, PUT y DELETE; aplica Bean Validation. |
| ClienteService -> ClienteRepository | Aplica unicidad y protección del historial, dentro de transacciones. |
| ClienteRepository -> clientes | Ejecuta consultas y persiste entidades JPA. |
| ClienteMapper -> ClienteResponse | Normaliza campos editables y entrega la respuesta. |
| AuditoriaAspect / GlobalExceptionHandler | Registra operaciones y traduce errores a respuestas HTTP. |

**Relaciones:** Cliente 1 -> N Venta; Venta 1 -> N DetalleVenta; Producto 1 -> N DetalleVenta. ventas.cliente_id es clave foránea. Eliminar un cliente no elimina sus ventas en cascada.

| Campo de Cliente | Restricción |
|---|---|
| id | Clave primaria autogenerada. |
| cedula | Única; la API requiere 10 dígitos. |
| nombre / apellido | Obligatorios; máximo 80 caracteres cada uno. |
| email | Obligatorio, formato válido, único; máximo 120 caracteres. |
| telefono / direccion | Opcionales; teléfono de 7 a 10 dígitos y dirección hasta 200 caracteres. |
| fecha_registro / activo | Administrados por el servidor; conservados al actualizar. |

## 3. Endpoints y reglas de negocio

Las solicitudes requieren X-API-KEY. En desarrollo, el valor predeterminado es grupo9-2026. Los ejemplos de la demostración usan datos ficticios.

| Método y ruta | Respuesta esperada |
|---|---|
| GET /api/clientes | 200: lista; admite filtro ?apellido=per. |
| GET /api/clientes/{id} | 200: cliente; 404 si no existe. |
| POST /api/clientes | 201 y Location; 400 por datos inválidos; 409 por duplicados. |
| PUT /api/clientes/{id} | 200; conserva metadatos; 404 inexistente; 409 por duplicados. |
| DELETE /api/clientes/{id} | 204 sin cuerpo; 404 inexistente; 409 si tiene ventas. |

## Reglas verificadas

1. La cédula no se repite.
2. El correo no se repite aunque cambien las mayúsculas; los correos nuevos se guardan en minúsculas con Locale.ROOT.
3. Actualizar con el propio correo es válido; usar el de otro cliente produce 409.
4. Un cliente con ventas no puede eliminarse, incluso si están anuladas.
5. PUT conserva ID, fecha de registro y estado activo.
6. Los datos inválidos se rechazan antes de persistirse.

## Excepciones y componentes transversales

RecursoDuplicadoException y ReglaNegocioException producen 409; RecursoNoEncontradoException produce 404; Bean Validation produce 400. El cuerpo uniforme contiene timestamp, status, error, mensaje, ruta y detalles. Sin clave válida, el interceptor devuelve 401. El aspecto registra crear, actualizar y eliminar en auditoría.

## Ejecución bloqueante y alternativa

JPA/JDBC bloquea el hilo mientras espera la base de datos. El proyecto integrado ofrece comprobantes asíncronos con @Async y CompletableFuture. Esa alternativa usa otro pool; no convierte JDBC en reactivo y las tareas en memoria no son una cola durable.

## Límite de la normalización

Las escrituras externas por SQL deben normalizar correos y revisar duplicados anteriores. No se modifican ni eliminan registros históricos. La cédula se valida por longitud y dígitos, no por dígito verificador.

## 4. Verificación y defensa

**Resultado local:** 69 pruebas ejecutadas, 0 fallos, 0 errores y 0 omitidas. La nueva clase ClientesIntegracionTest aporta 18 casos: CRUD y recarga JPA, duplicados, actualización, eliminación protegida en tres estados de venta, filtro, inexistencia, validación de seis campos e interceptor.

**Reproducir desde la raíz:**

```
.\mvnw.cmd -B -ntp verify
python scripts/demo-clientes.py --output target/clientes-http.json
```

La demostración inicia una API en un puerto local libre y una base H2 en disco temporal, ejecuta solicitudes reales, reinicia Java y vuelve a consultar el cliente. H2 usa WRITE_DELAY=0 para vaciar las escrituras antes del reinicio. Al terminar se elimina el cliente de prueba y se limpia el entorno temporal. Las respuestas se guardan en docs/evidencias/clientes-http.json.

La evidencia corresponde a H2 persistente. No acredita una ejecución contra PostgreSQL; el grupo debe verificar también su entorno integrado.

## Integración continua

El workflow existente se activa en feature/** y pull requests hacia develop. Configura Java 21, ejecuta clean verify y publica reportes Surefire. Incluye automáticamente las nuevas pruebas. Este pipeline compila y prueba; no se afirma un despliegue automático.

## Guion individual: unos 3 minutos

0:00-0:30: presentarse y delimitar el módulo.
0:30-1:05: explicar las capas y la relación Cliente-Venta.
1:05-2:10: demostrar CRUD, duplicado 409 y validación 400.
2:10-2:40: mostrar persistencia tras reiniciar y auditoría.
2:40-3:00: mostrar pruebas, commits y resultado de CI.

## Conclusiones y pendientes del equipo

La corrección impide duplicados por capitalización del correo sin cambiar el contrato HTTP. Como trabajo futuro: paginación, pruebas sobre PostgreSQL y migración controlada de correos históricos. El equipo debe integrar su informe, grabar la defensa con todos los integrantes y comprobar el acceso público al video. El cierre indicado de la actividad es el 15 de noviembre de 2026, 23:59.

## Referencias y material complementario

Enunciado Proyecto Grupal Unidad 1, Programación Avanzada, ESPE (2026). Repositorio del Grupo 9: código, README.md y CONTRIBUTING.md. Detalle técnico, diagramas y guion: docs/clientes.md; pruebas: ClientesIntegracionTest.java. Este documento registra el aporte del módulo, no la entrega en el aula virtual.